Ders 9 Genişletilmiş Notu: Gizli Anlamsal İndeksleme, Performans Ölçütleri ve Bilgi Çıkarımı

Ders 9 Genişletilmiş Notu: Gizli Anlamsal İndeksleme, Performans Ölçütleri ve Bilgi Çıkarımı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın. Altyazıda okunmayan slayt içerikleri (2×2 matris, "gold silver truck" tablosu, tekil değerler, belge vektörleri, NER etiketi) aynı dersin kısa notundan alınmıştır; o not yazılırken karelerden okunmuştu.

İçindekiler

- 1. Önceki Modellerden LSI'ye
- 2. LSI'nin Amacı ve Örnekleri
- 3. Ön İşlem ve Terim Ağırlıklandırma
- 4. Negatif Değerler ve Kullanım Alanları
- 5. Tekil Değer Ayrışımı: 2×2 Örnek
- 6. LSI Örneği: "gold silver truck"
- 7. Arama Motorları
- 8. Karışıklık Matrisi ve Hata Türleri
- 9. Duyarlılık ve Özgüllük
- 10. Bilgi Erişiminde Kesinlik, Anma ve F-Ölçüsü
- 11. Bilgi Çıkarımına Giriş
- 12. Belge Türleri
- 13. Düzenli İfadeler ve NLP'nin Katkısı
- 14. Adlandırılmış Varlık Tanıma
- 15. Güvenilir Sayfaları Belirleme
- 16. Dinamik Web Sayfaları ve Alışveriş Robotları
- 17. Bilgi Çıkarımında Performans
- 18. Şablon Tabanlı İlişki Çıkarımı

1. Önceki Modellerden LSI'ye

Bilgi erişiminde (information retrieval, IR) kullanıcı bir sorgu yazar. Arka planda çalışan model de sonuçları getirir. Geçen derste bu modellerin en eskilerinden başlanmıştı.

Boole modeli sorguya uyan belgeleri getiriyor ama sıralamıyordu. Genişletilmiş Boole modeli bir benzerlik formülüyle sıralama ekledi. OR sorgusunda kelime ağırlıklarının p. kuvvetleri toplanıp eleman sayısına bölünüyor ve p. dereceden kök alınıyordu. AND sorgusunda aynı işlem $1-w$ değerleriyle yapılıp sonuç 1'den çıkarılıyordu. Bulanık (fuzzy) modelde 1-0 yerine üyelik dereceleri vardı. Mix-min-max yönteminde her sorgu için minimum ve maksimum değerler, önceden deneylerle bulunmuş katsayılarla çarpılıyordu.

Bu modellerin ortak eksiği şuydu: yalnız kelimenin belgede geçip geçmediğine bakıyorlardı. Modele anlamsal bilgi katma düşüncesi cebirsel modelleri doğurdu. Bunların en önemlisi gizli anlamsal indeksleme (latent semantic indexing, LSI) yöntemidir. Yöntem gizli anlamsal analiz (latent semantic analysis, LSA) adıyla da geçer.

2. LSI'nin Amacı ve Örnekleri

LSI her belge için bir vektör, sorgu için de bir vektör oluşturur. Sorgu ile her belge arasındaki benzerlik skaler bir değer olarak ölçülür. Belgeler bu değere göre en benzerden en az benzere sıralanır ve kullanıcıya sunulur.

> **Hocanın vurgusu:** LSI anlamsal ilişkileri kullanır. Ortak kelime olmasa bile belgelerin anlamca birbirine benzediğini bulabilir.

Hocanın örneği şöyle: bir belge Atatürk'ten bahsediyor ama içinde "Türkiye" geçmiyor. Başka bir belgede "Türkiye" var ama "Atatürk" geçmiyor. LSI, Türkiye ile Atatürk arasındaki ilişkiyi belirler. "Türkiye" arandığında Atatürk ile ilgili sayfalar, "Atatürk" arandığında da Türkiye ile ilgili sayfalar gelebilir.

Yöntem yalnız terimin belgede geçip geçmediğine bakmaz. Terimle ilişkili başka terimleri de düşünerek sonuç kümesini genişletir. Kitapta iki örnek veriliyor:

- Associated Press haber veritabanında "Saddam Hüseyin" araması: Körfez Savaşı, BM yaptırımları ve ambargo makalelerinin yanında, Saddam Hüseyin adının geçmediği ama Irak hakkında bilgi veren makaleler de döner.
- "Tiger Woods" araması: ünlü golfçüyle ilgili makalelerin yanında, adının hiç geçmediği büyük golf turnuvası makaleleri de döner. Sistem, bu kişiyi arayan birinin golfle ilgilendiği çıkarımını yapar.

3. Ön İşlem ve Terim Ağırlıklandırma

3.1 Ön işlem

LSI, anlamca içerik taşıyan kelimeler (content words) üzerinde çalışmalıdır. Bu yüzden önce metin temizlenir:

- Başlıklar ve noktalama işaretleri atılır, bütün harfler küçültülür.
- Durak kelimeler (stop words) çıkarılır.
- Türkçe gibi sondan eklemeli dillerde ekler atılır. İngilizcede de çoğul ekleri gibi ekler vardır ama daha azdır. Türkçe için Snowball, Zemberek ya da Porter stemmer kullanılabilir.

Temizlenen içerik kelimelerinden terim-belge matrisi kurulur. Satırlar terimlere, sütunlar belgelere karşılık gelir (tersi de olabilir). Hücrede terimin belgede geçme sayısı ya da 1/0 bulunur.

Gerçek bir derlemde yüzlerce, binlerce belge vardır ve bir belgedeki kelime diğerlerinin çoğunda geçmez. Bu nedenle matris çok seyrektir (sparse). Seyreklik denince akla yumuşatma (smoothing) teknikleri gelebilir. Burada ise önce ham sayımlar yerine ağırlıklar kullanılır.

3.2 Yerel ağırlık

Bir belge içinde çok geçen kelime, bir kez geçen kelimeden daha anlamlıdır. Tek bir belge için yapılan bu ağırlıklandırmaya yerel ağırlık (local weighting) denir.

Örnek 1. Bir belgede "Türkiye" 10 kez, "yer" 1 kez geçiyor. Belgede toplam 11 kelime vardır.

- "Türkiye" için yerel ağırlık: $10/11 \approx 0{,}91$ (ondalık değer eklendi).
- "yer" için yerel ağırlık: $1/11 \approx 0{,}09$ (ondalık değer eklendi).

$10/11$ daha büyüktür, yani belge Türkiye ile daha ilişkilidir. Aynı işlem değerleri belli bir aralığa da oturtur, yani normalize eder.

3.3 Global ağırlık

Derlem düzeyinde durum tersine döner. Seyrek kullanılan kelime, yaygın kelimeden daha ilginçtir, çünkü daha ayırt edicidir. Buna global ağırlık (global weighting) denir.

Örnek 2. Derlemde 5 belge var. "A" kelimesi hepsinde, "B" kelimesi yalnız 2'sinde geçiyor. B "onkoloji" olsun. Bu durumda B, o iki belgenin onkolojiyle ilgili olduğunu söyleyebilir; A ise belgeleri ayırmaz. Hoca bu değeri tahtada $2 \cdot \log(5/2)$ olarak yazdı. Logaritma tabanı söylenmedi; 10 tabanında sonuç yaklaşık $0{,}80$ olur (ara adım eklendi).

Ders sonrası not: Hoca "global ağırlık TF-IDF'e karşılık gelir" dedi. Tam olarak yerel ağırlık TF'e, global ağırlık IDF'e karşılık gelir; ikisinin çarpımı TF-IDF'dir. Tahtadaki $2 \cdot \log(5/2)$ da bu çarpımdır.

3.4 Ağırlıklı matris

Eski modellerde terimin ağırlığı yalnız frekansıydı. LSA'da ağırlık üç çarpandan oluşur:

$$a_{ij} = L_{ij} \cdot G_i \cdot N_j$$

Burada L_{ij}, i. terimin j. belgedeki yerel ağırlığıdır. G_i, i. terimin global ağırlığıdır. N_j ise normalizasyon faktörüdür. Bu ağırlıklandırmayla matristeki sıfırlardan kurtulunur.

4. Negatif Değerler ve Kullanım Alanları

Terim-belge matrisine tekil değer ayrışımı uygulandıktan sonra matriste negatif değerler görülür. Oysa bir kelime bir belgede ya hiç geçmez ya da birkaç kez geçer; negatif sayım olmaz. Buradaki negatiflik sayım değil, anlamsal uzaklıktır:

- Pozitif değer: kelime ile belge ilişkilidir.
- Sıfır: nötr.
- Negatif değer: kelime ile belgenin içeriği birbirinden çok uzaktır.

Hocanın örneği: bir belge yaz tatilinden bahsediyorsa "onkoloji" kelimesiyle ilişkisi negatif bir değer olabilir.

LSA şu alanlarda kullanılır: bilgi erişimi, eş anlamlıları yakalama ("okul" ve "mektep"), çok anlamlılık (polysemy; yazılışı aynı, anlamı farklı kelimeler), arşivleme, belge sınıflandırma, özetleme ve filtreleme.

5. Tekil Değer Ayrışımı: 2×2 Örnek

Tekil değer ayrışımı (singular value decomposition, SVD), matristeki anlamsal bilgiyi çekmek için kullanılır. Gerçekte matris çok daha büyüktür ve değerleri ondalıklıdır; artı ve eksi de olabilir. Örnekte küçük bir matris kullanılıyor.

Örnek 3. Terim-belge matrisi şu olsun:

$$A = \begin{bmatrix} 4 & 0 \\ 3 & -5 \end{bmatrix}$$

Adım 1: $A^T A$. Transpoz, satır ve sütunların yer değiştirmesidir.

$$A^T A = \begin{bmatrix} 4 & 3 \\ 0 & -5 \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 3 & -5 \end{bmatrix} = \begin{bmatrix} 25 & -15 \\ -15 & 25 \end{bmatrix}$$

Adım 2: Özdeğerler (eigenvalues). $\det(A^T A - cI) = 0$ denklemi çözülür. I birim matris olduğu için cI matrisinin köşegeninde c bulunur.

$$A^T A - cI = \begin{bmatrix} 25 - c & -15 \\ -15 & 25 - c \end{bmatrix}$$

İçler dışlar çarpımı karakteristik denklemi verir:

$$(25 - c)^2 - 225 = 0 \quad \Rightarrow \quad c^2 - 50c + 400 = 0$$

Kökler $c_1 = 40$ ve $c_2 = 10$'dur. Azalan sırada sıralanırlar.

Adım 3: Tekil değerler (singular values). Tekil değerler özdeğerlerin karekökleridir: $s_1 = \sqrt{40} \approx 6{,}32$ ve $s_2 = \sqrt{10} \approx 3{,}16$. Köşegeninde büyükten küçüğe tekil değerler olan S kurulur. Sonra gerekeceği için tersi de bulunur.

$$S = \begin{bmatrix} \sqrt{40} & 0 \\ 0 & \sqrt{10} \end{bmatrix} \qquad S^{-1} = \begin{bmatrix} 1/\sqrt{40} & 0 \\ 0 & 1/\sqrt{10} \end{bmatrix}$$

Adım 4: Özvektörler (eigenvectors). Her özdeğer $(A^TA - cI)x = 0$ denkleminde yerine konur.

$c_1 = 40$ için:

$$\begin{bmatrix} -15 & -15 \\ -15 & -15 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad -15x_1 - 15x_2 = 0 \quad \Rightarrow \quad x_2 = -x_1$$

İki denklemi birbirinden çıkarmak $0 = 0$ verir ve işe yaramaz. Doğru yol, bir bilinmeyeni diğerinin cinsinden yazmaktır. Vektör $(x_1, -x_1)$ olur. Uzunluğu elemanların karelerinin toplamının kareköküdür:

$$\sqrt{x_1^2 + (-x_1)^2} = x_1\sqrt{2}$$

Her eleman uzunluğa bölünür: $(1/\sqrt{2},\ -1/\sqrt{2})$.

$c_2 = 10$ için aynı yoldan $15x_1 - 15x_2 = 0$, yani $x_2 = x_1$ bulunur. Birim vektör $(1/\sqrt{2},\ 1/\sqrt{2})$ olur.

Bu vektörler V matrisinin sütunlarıdır. İleride V^T de kullanılacaktır.

$$V = \begin{bmatrix} 0{,}7071 & 0{,}7071 \\ -0{,}7071 & 0{,}7071 \end{bmatrix}$$

Adım 5: U matrisi. $U = AVS^{-1}$ hesaplanır. Ara adım eklendi:

$$AV = \begin{bmatrix} 2{,}8284 & 2{,}8284 \\ 5{,}6569 & -1{,}4142 \end{bmatrix} \qquad U = AVS^{-1} = \begin{bmatrix} 0{,}4472 & 0{,}8944 \\ 0{,}8944 & -0{,}4472 \end{bmatrix}$$

Hoca A için "yalnız kelimeler ve belgeler arasındaki ağırlıkların tutulduğu matris", U için ise "kelimeler arasındaki anlamsal ilişkilerin de çıkarıldığı matris" diyor.

Adım 6: Geri dönüş. U, AVS^{-1} ile elde edildiği için A de USV^T ile geri elde edilir. Hoca sonucun yaklaşık çıktığını, farkın noktadan sonra alınan basamak sayısına bağlı olduğunu söylüyor. Tam değerlerle hesaplayınca A birebir geri gelir (python ile doğrulandı).

Matris $n \times n$ olsaydı n özdeğer ve n özvektör çıkardı. Pratikte bu işlemler elle yapılmaz. Kütüphanedeki LSA fonksiyonuna matris verilir, arka planda bu adımlar çalışır.

6. LSI Örneği: "gold silver truck"

Örnek 4. Üç belge ve bir sorgu var:

- d_1: "Shipment of gold damaged in a fire"
- d_2: "Delivery of silver arrived in a silver truck"
- d_3: "Shipment of gold arrived in a truck"
- Sorgu q: "gold silver truck"

Birinci belgede sorgudan bir kelime, ikincide üç kelime, üçüncüde iki kelime geçiyor.

> **Hocanın vurgusu:** "Burada üç kelime geçti, bu öne gelsin" diye düşünmeyin. Sadece kelimelerin geçip geçmediğine bakmıyoruz. Anlamsal olarak en yakın olanın gelmesini bekliyoruz.

Örnek küçük olduğu için durak kelimeler atılmadı; atılsaydı işlenecek kelime kalmazdı. Kök de alınmadı (“arrived” olduğu gibi kaldı). Terimler alfabetik sıralandı. Az belgeyle gerçekçi değer çıkmayacağı için ağırlıklandırma da yapılmadı, yalnız sayımlar kullanıldı.

Terim	d_1	d_2	d_3	q
a	1	1	1	0
arrived	0	1	1	0
damaged	1	0	0	0
delivery	0	1	0	0
fire	1	0	0	0
gold	1	0	1	1
in	1	1	1	0
of	1	1	1	0
shipment	1	0	1	0
silver	0	2	0	1
truck	0	1	1	1

Sonraki işlemler 2×2 örnekteki adımların aynısıdır. Hoca ara işlemleri yapmadan sonuçları gösterdi. Tekil değerler büyükten küçüğe $4{,}0989$, $2{,}3616$ ve $1{,}2737$’dir. Üç sütun olduğu için üç tekil değer ve üç özvektör vardır.

6.1 Rank seçimi

U’nun sütun sayısı çok fazla olabilir. Hepsiyle işlem yapmak gerekmez, en ayırt edici olanlar alınır. Kaç sütun alınacağını rank r belirler. Önce tekil değerlerin toplamı bulunur:

$$S_T = 4{,}0989 + 2{,}3616 + 1{,}2737 = 7{,}7342$$

Rank r için oran, ilk r tekil değerin toplamının S_T’ye bölümüdür:

r	Hesap	Oran
1	$4{,}0989/7{,}7342$	%53
2	$6{,}4605/7{,}7342$	%83,5
3	$7{,}7342/7{,}7342$	%100

Kabul edilen eşiği (örneğin %80) ilk aşan r seçilir. Sütun sayısı arttıkça %100’e yaklaşılır. Ama işlem maliyeti artar, kazanç ise noktadan sonra çok küçük değişimlerden ibaret kalır. Hoca bunu “attığın taş ürküttüğün kurbağaya değmez” diye özetliyor.

Bu örnekte $k = 2$ alındı: U’nun ilk 2 sütunu, S’nin 2×2’lik kısmı ve V’nin ilk 2 vektörü.

> **Ek bilgi (derste anlatılmadı):** Rank seçimi için birçok kaynak tekil değerlerin kendisi yerine karelerini kullanır. Çünkü tekil değerin karesi, o bileşenin açıkladığı varyansla orantılıdır. Dersteki oran ise doğrudan tekil değerlerle hesaplanıyor.

6.2 Belge ve sorgu vektörleri

Başta d_1 vektörü yalnız kelime sayımlarından oluşuyordu. Artık belgeler V_k’nın satırlarıyla, yani iki boyutlu vektörlerle temsil edilir. Bu değerler yalnız belgedeki kelimelerden değil, kelimelerin diğer kelimelerle birlikte geçmesinden de çıkar. Dört basamakla (python ile hesaplandı):

- $d_1 = (-0{,}4945;\ 0{,}6492)$
- $d_2 = (-0{,}6458;\ -0{,}7194)$
- $d_3 = (-0{,}5817;\ 0{,}2469)$

Sorgu da aynı uzaya taşınmalıdır. Sorgu vektörünün transpozu, indirgenmiş U_k ve indirgenmiş S_k'nın tersiyle çarpılır:

$$q_k = q^T U_k S_k^{-1} = (-0{,}2140;\ -0{,}1821)$$

6.3 Kosinüs benzerliği

Benzerlik, vektörlerin iç çarpımının uzunluklarının çarpımına bölümüdür:

$$\text{sim}(q, d) = \frac{q \cdot d}{|q|\,|d|}$$

Ara adımlar eklendi, $|q| \approx 0{,}2810$:

Belge	$q \cdot d$	$\lvert d\rvert$	sim
d_1	$-0{,}0124$	$0{,}8161$	$-0{,}05$
d_2	$0{,}2692$	$0{,}9667$	$0{,}99$
d_3	$0{,}0795$	$0{,}6319$	$0{,}45$

Sıralama $d_2 > d_3 > d_1$ olur. Sistem kullanıcıya önce d_2'yi, sonra d_3'ü, en son d_1'i getirir. İki boyutlu uzay grafiğinde de sorguya en yakın belge d_2'dir.

Hoca burada "gold" kelimesinin o kadar etkili olmadığını, "silver truck" kısmının öne çıktığını söylüyor. Bunun nedenini bilmediklerini, cümlelerin zaten yalnız örnek için yazıldığını ekliyor.

7. Arama Motorları

Hoca bu kısmı yüzeysel geçti, çünkü sürekli değişiyor ve bilgiye kolay ulaşılıyor.

Önceleri web katalogları vardı. Kataloglar elle seçilmiş sitelerin tanımlarında arama yapar ve siteleri hiyerarşik kategorilere ayırır. Arama motorları ise bütün sitelerin bütün sayfalarının içeriğinde arar ve sonuçları skora göre sıralar.

Google, PageRank algoritmasını kullanır. PageRank değeri ne kadar yüksekse site o kadar popülerdir. Puan; başlıklar, bağlantı metinleri (anchor text), sayfaya verilen linkler ve sayfadan giden linkler gibi girdilerle karışık hesaplarla çıkar. Hoca formülü slayta koymadığını söylüyor.

Temel fikir şudur: bir sayfa sürekli başka sayfalara link veriyorsa çok bilgi içermiyordur, yalnız yönlendiriyordur. Herkes bir sayfaya link veriyorsa o sayfanın PageRank'i yüksektir. En kaba hâliyle, bir siteye ne kadar farklı siteden link verilmişse o site o kadar önemlidir.

> **Ek bilgi (derste anlatılmadı):** PageRank'in yaygın biçimi $PR(p) = \frac{1-d}{N} + d\sum_{q \to p} \frac{PR(q)}{L(q)}$ şeklindedir. Burada d sönüm katsayısı, N sayfa sayısı, $L(q)$ de q sayfasından çıkan link sayısıdır. Yani çok link veren bir sayfa, verdiği her linke daha az puan aktarır.

Hocaya göre akıllı bir arama motorunda şunlar olmalıdır:

- **Eş anlamlılar:** "restoran" aranınca lokanta, kafe, pub da gelmeli.
- **Kısaltmalar:** "PRC" (People's Republic of China) için "China" da gelmeli.
- **Çok anlamlılık:** Apple şirketi ile elma meyvesi ayrılmalı. Elma kurusundan bahsederken şirket bilançosu görünmemeli. "bat" hem beyzbol sopası hem yarasa demektir.
- **Sorgudaki kelime sırası:** İnsan önemli olanı öne yazar. Öndeki kelimeye öncelik verilmeli.
- **Kullanıcı davranışı:** Aynı sorguda herkes üçüncü sıradaki linke gidiyorsa benzer sorgularda o link daha yukarıda gelmeli.
- **Sorgu genişletme:** "Yıldız Teknik Üniversitesi" aranınca eski adı "Yıldız Üniversitesi" geçen sayfalar da gelmeli.
- **İmla denetimi:** "Bunu mu demek istediniz?"

- **Kaynak güvenilirliği:** Tamamen yanlış bilgilerle dolu bir sayfa da indekslenir ve aramada çıkabilir. Bu yüzden indekslemede içeriğin doğruluğu da kontrol edilmeli.

8. Karışıklık Matrisi ve Hata Türleri

Sistem tasarlamak yetmez, doğruluğunu da ölçmek gerekir. "10 verdim, 5 aldım, %50 doğrudur" demek yeterli değildir. Bunun için duyarlılık (sensitivity), kesinlik (precision), anma (recall), doğruluk (accuracy) ve F-ölçüsü (F-measure) gibi ölçütler vardır. Duyarlılık ile anma aynı şeydir; bilgi erişiminde "recall" sözcüğü tercih edilir.

Hocanın tablosu hasta insanlar ve test sonuçları üzerinedir:

	Test pozitif	Test negatif
Gerçekte hasta	doğru pozitif (TP)	yanlış negatif (FN)
Gerçekte sağlam	yanlış pozitif (FP)	doğru negatif (TN)

- Hasta birine test pozitif çıkarsa durum doğrudur: TP.
- Hasta birine test negatif çıkarsa, yani "sağlamsın" denirse durum hatalıdır: FN.
- Sağlam birine test pozitif çıkarsa, yani "hastasın" denirse durum hatalıdır: FP.
- Sağlam birine test negatif çıkarsa durum doğrudur: TN.

Hoca bu kavramların sık karıştırıldığını, kendisinin de bu tabloyu akılda tutarak hatırladığını söylüyor. Yazılımdan bir örnek veriyor: hatalı yazılmış kod teste girer ve test hatayı bulursa TP olur. Kodun hatalı olduğu bilinir ama testler hatayı bulamazsa FN olur.

Amaç TP ve TN'yi büyütmek, FP ve FN'yi küçültmektir. Her şeyin köşegene oturup diğer hücrelerin sıfır kalması teoride belki mümkündür, pratikte zordur.

FP'ye Tip I hata (type I error), FN'ye Tip II hata (type II error) denir. Hangisi daha tehlikelidir?

- Sağlam birine "hastasın" denirse morali bozulur. Birkaç gün sonra bir şey olmadığını görür ve yoluna devam eder.
- Hasta birine "hasta değilsin" denirse belki morali yükselir. Ama hastalık sürer ve durum eninde sonunda kötüleşir.

  Hocanın vurgusu: En kötü hata FN'dir. Ölümcül sonuçlar FN'dedir. FP yerine FN'yi en aza indirmek çok daha iyidir.

  Ders sonrası not: Bu değerlendirme hastalık teşhisi için geçerlidir. Hangi hatanın daha pahalı olduğu probleme göre değişir. Örneğin önemli bir e-postayı istenmeyen posta sayan sistemde FP daha maliyetli olabilir.

Bu matrise karışıklık matrisi (confusion matrix) denir. Türkçede hata matrisi ya da karmaşıklık matrisi de denir. Genelde 2×2 gösterilir, ama öyle olmak zorunda değildir.

9. Duyarlılık ve Özgüllük

Hoca kavramları iki polis köpeğiyle anlatıyor. Birinci köpek yalnız kokaini tanımak için eğitilmiştir, başka kokuyu ayırt edemez. Alanı dardır, kaçırma ya da karıştırma şansı azdır. İkinci köpek kokain, eroin, esrar gibi farklı kokular için eğitilmiştir. Yelpazesi geniştir, ama içlerinden belirli birini bulması istendiğinde yanılabilir. Hoca özgüllüğü (specificity) birinci köpeğe, duyarlılığı ikinci köpeğe benzetiyor.

$$\text{Özgüllük} = \frac{TN}{TN + FP} \qquad \text{Duyarlılık} = \frac{TP}{TP + FN}$$

Özgüllük tablonun sağlam satırına bakar: sağlam olanların ne kadarına doğru olarak "sağlam" denmiş? Hoca bunu da bir örnekle anlatıyor: köpek temiz bir nefesi tanır ve kişiye sağlam der. Ağız kokusunun nedeni diş ya da mide olabilir, ama köpek bunları bilmez, yalnız sağlam olup olmadığını bulur. Duyarlılık ise hasta satırına bakar: gerçekten hasta olanların ne kadarı yakalanmış?

Hoca şunu da ekliyor: geniş yelpazeli bir test farklı hastalıkları tanıyabilir ama sağlam bir insanı daha az tanıyabilir.

> **Ders sonrası not:** Köpek benzetmesi formüllerle birebir örtüşmüyor. Birinci köpek için söylenen "kokaini kaçırmaz" sözü aslında duyarlılığı anlatır. Formülleri esas alın: duyarlılık gerçek pozitiflere, özgüllük gerçek negatiflere bakar.

Başka ölçütler de vardır:

- Yanlış negatif oranı $= 1-$ duyarlılık.
- Yanlış pozitif oranı $= 1-$ özgüllük.
- Pozitif ve negatif kestirim değerleri (positive/negative predictive value).

Hoca kestirim değerlerinin tıp çalışmalarında önemli olabileceğini, bilgisayar alanındaki akademik çalışmalarda pek kullanılmadığını söylüyor.

> **Ders sonrası not:** Hoca bir yerde iki oranı da "false negative rate" diye söyledi. İkincisi "false positive rate" olmalıdır: yanlış pozitif oranı $= 1-$ özgüllük.
>
> **Ek bilgi (derste anlatılmadı):** Hocanın adını andığı doğruluk (accuracy) ölçütü, bütün örnekler içinde doğru kararların oranıdır: $\frac{TP+TN}{TP+TN+FP+FN}$.

10. Bilgi Erişiminde Kesinlik, Anma ve F-Ölçüsü

Bilgi erişiminde tablo farklıdır. Web'de bir sorguyla ilgili (relevant) belgeler vardır, bir de sorguya karşılık getirilen (retrieved) belgeler. Aradığımız kısım bu iki kümenin kesişimidir. Arama motoru ne kadar iyi tasarlanırsa bu iki küme o kadar örtüşür.

	İlgili	İlgisiz
Getirilen	ilgili ve getirilen	ilgisiz ama getirilen
Getirilmeyen	ilgili ama getirilmeyen	ilgisiz ve getirilmeyen

$$\text{Kesinlik (precision)} = \frac{|\text{ilgili} \cap \text{getirilen}|}{|\text{getirilen}|} \qquad \text{Anma (recall)} = \frac{|\text{ilgili} \cap \text{getirilen}|}{|\text{ilgili}|}$$

Kesinlik, getirilenlerin ne kadarının doğru olduğunu söyler. Anma, gelmesi gerekenlerin ne kadarının geldiğini söyler. Hoca, anmanın özgüllüğe karşılık gelmediğini, ikisinin karıştırılmaması gerektiğini vurguluyor.

> **Ders sonrası not:** Hoca precision için "duyarlılık" dedi. Türkçe kaynaklarda precision çoğunlukla "kesinlik", recall ise "duyarlılık" ya da "anma" diye geçer. Bu notta precision = kesinlik, recall = anma kullanıldı.
>
> **Hocanın vurgusu:** İki bilgi de önemlidir ama ayrı ayrı kullanmak sistemin performansı konusunda yanıltır. Bu yüzden ikisinin harmonik ortalaması olan F-ölçüsü kullanılır.

$$F = \frac{2 \cdot P \cdot R}{P + R}$$

F değeri 1'e ne kadar yakınsa sistem o kadar iyidir. $F = 1$, başarının %100 olması demektir.

> **Ders sonrası not:** Hoca paydaki 2 çarpanının "denemeler sonucu bulunduğunu" söyledi. Bu çarpan harmonik ortalamanın tanımından gelir: $F = \frac{2}{1/P+1/R} = \frac{2PR}{P+R}$.

Karışıklık matrisi ikiye ikiyse bir kesinlik ve bir anma hesaplanır. Sınıf sayısı 2'den fazlaysa N kesinlik ve N anma hesaplanıp ortalamaları alınır. Hocanın örneği yazar tanımadır: 100 belge ve 10 yazar var, belgelerin doğru yazara atanıp atanmadığı bulunmak isteniyor. Bu problemde 10 sınıf vardır.

11. Bilgi Çıkarımına Giriş

Bilgi erişimi bitti, sırada bilgi çıkarımı (information extraction, IE) var. Hoca konunun çok büyük olmadığını, birçok parçasının önceki slaytlarda geçtiğini söylüyor. İşin çoğu da kendi yazılan kodlarla yapılır; hazır bir araç yoktur.

IR ile IE'nin ilişkisi şöyledir: sorgu gönderilir, belgeler gelir. Belgeler okunup aranan bilgi bulunur. O bilgiye ulaşma işi bilgi çıkarımıdır. Örnek: Uganda'nın başkenti bilinmiyor. "Uganda başkent" diye aranır, gelen belgelere hızla bakılır ve başkent bulunur. Bu tür soruların cevabı yazılan kodlarla çıkarılabilir.

> **Ders sonrası not:** Altyazıda başkent için "Namajigoro" geçiyor. Uganda'nın başkenti Kampala'dır.

Aynı iş soru cevaplamaya (question answering) da benzer: "Uganda'nın başkenti neresi?" diye sorulur, sistem cevabı döndürür.

> **Hocanın vurgusu:** IR, IE ve soru cevaplama iç içedir. IR iyi çalışırsa IE iyi çalışır, IE iyi çalışırsa soru cevaplama başarılı olur.

Tanım. Bilgi çıkarımı; yapılandırılmış, yarı yapılandırılmış ya da yapılandırılmamış verilerden, belirli şablonlar ya da yazılan kodlarla istenen bilgiyi bulup çıkarmaktır.

Message Understanding Conference (MUC) yarışmalarında sistemler verilen platformda sorulara en doğru cevabı getirmek için yarışır. Hoca bunun ABD'nin devlet politikasıyla da ilgili olduğunu söylüyor: Web'i izleyip kendisiyle ilgili her şeyi otomatik çıkaracak uygulamalar.

> **Ders sonrası not:** Hoca MUC'un 1990'lardan beri her yıl yapıldığını söyledi. MUC serisi 1987'de başladı ve 1990'ların sonunda sona erdi.

Uygulama örnekleri:

- Haber makalelerinden ve tweetlerden terör olaylarını saptamak.
- Şirket birleşmelerini ve ayrılmalarını haberlerden önceden çıkarıp borsada kullanmak.
- İş ve işçi bulma: farklı ilan sitelerinden, örneğin LinkedIn'den, kriterlere uyan kişileri iletişim bilgileriyle getirmek.
- Farklı sitelerde satılan ve kriterlere (örneğin fiyat) uyan ürünleri getirmek.
- Farklı platformlardaki seminer duyurularını (örneğin fotoğrafçılık) toplamak.
- Kiralık ve satılık daire ilanlarını taramak.

Hepsinin ortak özelliği, birden çok yerden istenen özelliklere sahip bilgiyi toplayıp getirmektir.

12. Belge Türleri

12.1 Yarı yapılandırılmış belge

Örnek e-postadır. Konu, tarih, gönderen ve gönderenin adresi gibi alanların yeri bellidir, kolayca çekilir. Sorun, yapılandırılmamış gövdededir.

Hocanın örneği bir yazılım geliştirici ilanıdır. Gövdede aranan özellikler yazılıdır: deneyim, C bilgisi, 5 yıl deneyim, platform (DOS, OS/2, Unix). Bu anahtar kelimeler düzenli ifadeler ve dile özgü NLP özellikleriyle çekilir. Çekilen bilgiler veritabanı alanlarına (dil, platform, deneyim, ilan numarası, ülke) yerleştirilir. Aramalar da bu veritabanından yapılır.

12.2 Yapılandırılmamış belge

En zor çalışma alanı budur. Belge düz bir paragraftır ve sorular bu paragraftan cevaplanır. Örnek cümle (adlar burada A ve B ile gösterildi):

"21 yaşındaki inşaat işçisi A, evine dönerken para meselesi yüzünden tartıştığı arkadaşı B tarafından bıçaklanarak öldürüldü."

Soru	Cevap
Kim öldürüldü?	A
Kim öldürdü?	B

Soru	Cevap
A kaç yaşındaydı?	21
İşi neydi?	İnşaat işçisi
Neden öldürüldü?	Para meselesi
Nerede öldürüldü?	Evine dönerken
B kimdi?	Arkadaşı
Ne ile öldürüldü?	Bıçakla

Bu cevapları çıkarmak kolay değildir. Yaş bilgisinin kime ait olduğu baştan belli değildir. Ama "tarafından bıçaklanarak öldürüldü" kalıbında "tarafından" kelimesinin önündeki adlandırılmış varlık öldüreni verir. "-lanarak" eki alan kelimenin kökü de suç aletini gösterir. Bunlar için NLP gerekir. 5N1K sorularının hepsini cevaplamak büyük bir problemdir. Cümleler karmaşık olabilir, içinde birçok kişi geçebilir. Sözdizimsel çözümleme ve sözcük türü etiketleri (POS tags) gerekir.

12.3 Yapılandırılmış belge

En kolay iş budur. Genelde web sayfalarıdır. Amazon'un kitap sayfası örnek verilebilir: bütün kitaplar aynı biçimde verilir. Hangi HTML etiketleri arasında hangi bilginin olduğu bellidir: kitap adı, yazar, fiyat, ciltli ya da ciltsiz, baskı. Fiyat karşılaştırma siteleri bunu kullanır. Özel bir NLP aracına gerek yoktur.

13. Düzenli İfadeler ve NLP'nin Katkısı

13.1 Düzenli ifadeler

Metinden bilgi çıkarmanın en kolay yolu düzenli ifadelerdir (regular expression), ama her bilgi bu yolla çıkmaz. Adres ve telefon bilgileri çıkarılabilir. Doğum yeri ve tarihi de çıkarılabilir, çünkü insanlar bunu sınırlı sayıda biçimde yazar:

- "80 doğumluyum"
- "1980 doğumluyum"
- "1980 yılında İstanbul'da doğdum"
- "İstanbul'da 1980 yılında doğdum"

Bu biçimler tek bir düzenli ifadenin içine alınabilir. Oysa yukarıdaki cinayet cümlesi düzenli ifadelerle kolayca çözülmez.

Hocanın önerdiği sıra şöyledir: yapılabiliyorsa düzenli ifade, yapılabiliyorsa standart şablon, olmuyorsa NLP. Bunların hiçbiriyle çözülmüyorsa bilgi çıkarılamaz, okuyarak bulmak gerekir.

Hoca bir de düşüncesini paylaşıyor. Bu konuda çalışmadığını ve araştırma yapmadığını belirterek, derin öğrenmeyle metinler ve soru-cevap çiftleri verilip bunun öğretilebileceğini söylüyor. Ama milyonlarca belge ve etiket gerekir, etiketleme de zaman ve para ister.

13.2 NLP'nin katkısı

- **Sözcük türleri:** Bir sorunun cevabı isim, fiil ya da sıfat olabilir. "Hava nasılmış?" sorusu için metindeki sıfatlar (güzel, yağmurlu) aranır.
- **Sözdizimsel çözümleme:** Kim vurdu, neden vuruldu? Özel isimler, isim öbekleri ve fiil öbekleri bulunur.
- **Anlamsal sınıflandırma:** WordNet kullanılabilir. "Öldürmek" eş anlamlı kümesinde (synset) "kill", "murder" gibi ölümle ilişkili kelimeler vardır; bunlar birlikte değerlendirilir.
- **Adlandırılmış varlıklar:** Metindeki kişi, yer, kurum, tarih ve zamanlar bulunur.

Bunlar için hazır ya da kendi yazılan NLP modülleri kullanılır. Hepsi bir araya geldiğinde iyi bir bilgi çıkarım sistemi tasarlanmış olur.

Örnek: “ye-” fiilinin nesneleri. Yenilebilen şeyler bulunmak isteniyor. İnsan birkaçını sayar, ama çoğunu unutur. Arama motoru bu soruya cevap verebilir mi? “yerken” aranınca şunlar gelir: yarım döner, kraker, zeytin, mantar. Ama hatalı sonuçlar da gelir:

- “family yerken” gibi bir sonuçta aile yenilebilen bir şey sanılır.
- “Yerken” büyük harfle yazılmışsa soyadıdır. “ye-” fiili büyük harfle yazılmaz, bu kuralla elenebilir.
- “Kiraz Yerken” bir kişinin adı ve soyadı olabilir. Kiraz yenilen bir şeydir ama burada özel isimdir. Bu durum adlandırılmış varlık tanımayla çözülür.

14. Adlandırılmış Varlık Tanıma

Adlandırılmış varlık tanıma (named entity recognition, NER) metindeki varlık adlarını bulur. Temel üç varlık kişi (person), yer (location) ve kurumdur (organization). Ardından para birimi, tarih ve zaman gelebilir. Varlıklar alt türlere de ayrılabilir: yer için ülke, şehir ve belde gibi hiyerarşik etiketler kullanılabilir.

Etiketleme şöyle görünür:

```
<PER>Jane Brown</PER> saw a cat in <LOC>London</LOC>
```

Etiket açılır ve kapatılır. Arasında kalan ifade o türdendir.

14.1 Kural tabanlı yöntem

En ilkel yöntemdir. “New York yerdir”, “California yerdir” gibi her şey tek tek yazılır. “Mr.” ile başlıyorsa kişi, “Corporation” ya da “Co.” geçiyorsa şirket denir. Bu listede olanlar kaçmaz, ama her şeyi tek tek vermek çok maliyetlidir. Yöntem artık geride kaldı.

14.2 Şablon tabanlı yöntem

En basit yol şablon hazırlamaktır.

- **“unvan + kişi”:** Elde unvan sözlüğü vardır. Metinde “President” geçerse arkasından gelen kişidir: George Bush. Ardından nokta olmadan büyük harfle devam eden kelime, aynı ismin ikinci ya da üçüncü parçasıdır.
- **“kişi, the unvan of kurum”:** “Fred Martin, the CEO of XYZ Corp.” cümlesinde “the” ve “of” eşleşir. CEO unvan, Fred Martin kişi, XYZ Corp. kurumdur.

Her cümle şablon üzerinde pencere kaydırılarak (window) taranır. Şablon otururs boşluklardaki kelimeler etiketlenir. Asıl önemli iş bu şablonları çıkarmaktır. Yer, telefon numarası, tarih, URL ve e-posta adresi için de şablonlar yazılabilir, bunlar birleştirilerek yenileri oluşturulabilir.

Şablonla bulunan sonuçlara ağırlık verilir. Her şablon dilde aynı sıklıkta görülmez. Daha sık yazılan şablonun ağırlığı daha yüksektir. Sonuçlar bu ağırlıklar dikkate alınarak sunulur. Örnek çıktılar: Kodak, Apple Computer, Cupertino.

14.3 Koşullu rastgele alanlar

Koşullu rastgele alanlar (conditional random fields, CRF) şablonların daha ileri bir biçimidir. Hoca “Doktor Banu Diri” örneğiyle anlatıyor: solunda “Doktor” ya da “Dr.” varsa, sağında da belli bir şey varsa, ortadakinin isim olma olasılığı hesaplanır. Durumlar (state) arasında geçiş olasılıkları vardır. Örneğin “Doktor” geçişinde kişi olma olasılığı 0,98 olabilir. Yeni bir metin geldiğinde önünde ve arkasında bu kelimeler varsa en yüksek olasılıklı etiket verilir.

> **Ders sonrası not:** Hoca CRF’in saklı Markov modellerine (hidden Markov model) benzediğini, onları kullandığını söyledi. İkisi akrabadır ama aynı değildir. HMM üretici (generative), doğrusal CRF ise ayırt edici (discriminative) bir modeldir. CRF arka planda HMM çalıştırmaz.

14.4 Derin öğrenme

Derin öğrenmede olasılıklar elle tanımlanmaz. Etiketli veri verilir, sistem etiketlerin arasına ne geldiğini öğrenir. Örneğin hep "Doktor"dan sonra kişi geldiğini görür ve bundan sonra böyle etiketler.

Başarı %100 değildir. NER birçok araçta İngilizce ve Türkçe için vardır. Hoca Türkçe kaynaklar için iki firmaya bakılmasını öneriyor: Starlang NLP araçları ve veri setleri paylaşıyor. Loodos, derin öğrenme ve BERT kullanan Türkçe dil modelleri geliştirip kullanıma açtı.

15. Güvenilir Sayfaları Belirleme

Bilgi çıkarırken önemli olan, sayfalara ne kadar güvenilebileceğidir. Her sayfada doğru bilgi yoktur. Yöntem şöyledir:

1. Doğruluğu bilinen örnek çiftler seçilir: Ankara–Türkiye, Paris–Fransa, Türk lirası–Türkiye, dolar–ABD.
2. Bu çiftler arama motoruna gönderilir, ilgili sayfalar gelir. Bu sayfalardan sonuç şablonları çıkarılır: başkent şablonu, para birimi şablonu.
3. Şablonlar arama motoruna gönderilir, şablona uyan sayfalar aranır.
4. Gelen cevaplar veritabanıyla karşılaştırılır. Örneğin başkent şablonundan "Türkiye'nin başkenti İstanbul" gelirse veritabanı bunu doğrulamaz.
5. Yanlış bilgi içeren sayfalar elenir. Aramalar yalnız kalan sayfalar üzerinden yapılır. Arama motoru bu sayfaları indekslemeyi de keser.

16. Dinamik Web Sayfaları ve Alışveriş Robotları

Web sayfalarından bilgi çıkarmak kolaydır, çünkü HTML kurallı bir dildir ve etiketler arasında tekrar eden kalıplar vardır. Örneğin bir tablonun başlangıcı ve bitişi bellidir. Her satır aynı kalıpla tekrar eder.

Alışveriş robotlarına örnek olarak Froogle, mySimon ve BookFinder veriliyor. Bir alışveriş robotu şöyle çalışır:

1. Her satıcı site için bilgi çıkarma mekanizması kurulur.
2. Kullanıcının sorgusu alınır.
3. Sorgu her siteye tek tek gönderilir, sonuç sayfaları alınır.
4. Sonuç sayfalarından bilgi çıkarılıp veritabanına kaydedilir.
5. Sonuçlar istenen ölçüte göre sıralanıp kullanıcıya döndürülür.

17. Bilgi Çıkarımında Performans

Kesinlik, anma ve F-ölçüsü burada da kullanılır, yalnız tanımlar uyarlanır:

- N: belgelerdeki doğru cevap sayısı.
- E: sistemin çıkardığı toplam cevap sayısı.
- C: sistemin çıkardığı doğru cevap sayısı.

$$\text{Kesinlik} = \frac{C}{E} \qquad \text{Anma} = \frac{C}{N}$$

Mantık aynıdır. Kesinlik, getirdiklerimin ne kadarı doğru sorusunu cevaplar. Anma, getirmem gerekenlerin ne kadarını getirdim sorusunu cevaplar. F-ölçüsü yine ikisinin harmonik ortalamasıdır.

18. Şablon Tabanlı İlişki Çıkarımı

Bilgi, ilişki şablonlarıyla da toplanabilir.

"Tüm X'ler Y'dir" ilişkisi. Örnek: "bütün kuşlar kanatlıdır".

1. “kuş” ve “kanat” birlikte arama motoruna gönderilir. Şu kalıplar gelir: “kuş ve diğer kanatlılar”, “kuşlar ve diğer kanatlılar”, “kuş ve benzeri kanatlılar”, “kuş veya diğer kanatlılar”, “kuş türü olan kanatlı”.
2. Bu kalıplar aranarak aynı ilişkiye sahip başka ikililer bulunur, frekansları çıkarılır.
3. Bulunan ikililer elle işaretlenir. Bazıları uyar: teçhizat–malzeme, protein–gıda, komünizm–ideoloji. Bazıları uymaz: kız–sıfat gibi bir ikili bu ilişkiye uymaz.

“X’in yeri Y’dir” ilişkisi. “Beşiktaş” ve “İstanbul” birlikte aranır. Cümlelerden “İstanbul ili sınırları içerisinde bulunan X” gibi kalıplar çıkar. Bu kalıplarla örtüşen başka ikililer de bulunabilir.

> **Hocanın vurgusu:** Bilgi çıkarımında sıra şudur. Önce doğru sayfalarda arama yaptığından emin ol. Yapılabiliyorsa düzenli ifade yaz. Yetmezse NLP’den yardım al: sözcük türleri, adlandırılmış varlıklar, sözdizimsel dizilimler. Ayrıca şablonlar oluştur, bu şablonların geçtiği belgelerden veri topla ve soruları bu verilerle cevapla.

Dersin Toparlaması

Ders iki büyük parçadan oluştu. İlk parçada önceki bilgi erişimi modellerinin eksiği olan anlamsal bilgi, LSI ile ele alındı. Terim-belge matrisi ağırlıklandırıldı, SVD ile ayrıştırıldı ve rank k ile indirgendi. Belgeler ile sorgu aynı uzaya taşınıp kosinüs benzerliğiyle sıralandı. Bu sayede ortak kelime sayısı değil, anlamsal yakınlık belirleyici oldu. Ardından arama motorlarının özellikleri ve sistem başarısını ölçen karışıklık matrisi, duyarlılık, özgüllük, kesinlik, anma ve F-ölçüsü işlendi.

İkinci parçada bilgi çıkarımı anlatıldı. Belge yapısı zorlaştıkça düzenli ifadelerden NLP araçlarına geçildi. NER, güvenilir sayfa belirleme ve şablon tabanlı ilişki çıkarımı bu parçanın ana yöntemleriydi. Hoca, IR iyi olursa IE’nin, IE iyi olursa soru cevaplamanın iyi çalışacağını belirterek konuları bağladı. Ders başında anılan metin sınıflandırma bu kayıtta anlatılmadı.

Ders Sonrası Notlar

Metin içindeki notlara ek olarak:

> **Ders sonrası not:** SVD anlatımında hoca önce “tekil değerlerden özdeğerler elde edilir” dedi. Hesapta sıra terstir: önce özdeğerler, sonra onların karekökü olan tekil değerler bulunur.
>
> **Ders sonrası not:** Karakteristik denklem için hoca köklerin “toplamının eksi 50” olduğunu söyledi. $c^2 - 50c + 400 = 0$ denkleminde çarpım 400, toplam $+50$’dir.
>
> **Ders sonrası not:** Özvektör adımında hoca $c_1 = 40$ için “$x_2 = x_1$” dedi. Doğrusu $x_2 = -x_1$’dir; hocanın devamında yazdığı $(x_1, -x_1)$ vektörü de bunu gösterir. Ayrıca bir yerde “dörde dörde” dedi; matris 2×2’dir.
>
> **Ders sonrası not:** Rank anlatımında hoca birinci oran için “0,98” dedi; değer $4{,}0989$’dur. “%80, %85, 0,75” gibi rakamları da yalnız varsayım olarak söyledi. Oranlar birikimli olduğu için r arttıkça büyür ve $r = 3$ her zaman %100’dür. Bu örneğin gerçek değerleri %53, %83,5 ve %100’dür.
>
> **Ders sonrası not:** Sorgu örneğinde altyazıda önce “gold silver” geçiyor, hoca hemen “gold silver truck” diye düzeltiyor. d_1 altyazıda “to meet in a fire” diye geçer; doğrusu “damaged in a fire”.
>
> **Ders sonrası not:** Altyazıdaki bozuk yazımlar: “laten semantik indexing” ve “LST” LSI, “portrait stammer” Porter stemmer, “aygın value” eigenvalue, “Bull’un model” Boole modeli, “MOOC” MUC, “Ludus” Loodos, “Frogle” Froogle, “but” bat, “Kumpertino” Cupertino, “Jed/Jack Brown” Jane Brown. “Sinet” adı büyük olasılıkla CNET’tir, kesin değildir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.